Relatório desafio DEVS - Fernando Corrêa Gambôa Pereira dos Santos

1. O que foi feito?

Construí a API da loja de plantas Folhagem & Cia com Node.js, Express, TypeScript, Prisma e PostgreSQL, rodando em Docker, seguindo a mesma organização em camadas do CRUD de usuário que já vinha no repositório (rotas, controller e repository). O código está na branch desafio/fcgps do meu repositório, criado a partir do template.

Modelo de dados: Criei o enum Luminosidade (SOMBRA, MEIA_SOMBRA, SOL_PLENO) e o model Planta, com id, nome, especie, preco (em centavos), estoque, luminosidade, petFriendly, regaACadaDias, createdAt e updatedAt. A migration create_planta_model foi gerada pelo Prisma e está commitada.

Rotas obrigatórias. Implementei as sete:

- POST /plantas cadastra uma planta (201).
- GET /plantas lista as plantas, com os filtros ?luminosidade= e ?petFriendly=, sozinhos ou juntos (200).
- GET /plantas/:id busca uma planta (200).
- PATCH /plantas/:id edita só os campos enviados (200).
- DELETE /plantas/:id remove uma planta (204).
- POST /plantas/:id/venda recebe a quantidade e diminui o estoque (200).
- GET /plantas/estoque/valor-total devolve valorTotalEmCentavos e totalDeVasos (200).

Regras de negócio. Implementei as sete: todos os campos obrigatórios no cadastro; preço maior que zero; regaACadaDias pelo menos 1; estoque não negativo, mas podendo ser zero; luminosidade só com os três valores do enum; 400 com mensagem clara para dados inválidos (no cadastro, na edição, nos filtros e na venda); 404 para id inexistente; e venda que nunca deixa o estoque negativo, respondendo 400 sem alterar nada quando não há estoque suficiente. Com a loja vazia, o valor total responde 200 com os valores zerados. Duas compras ao mesmo tempo. O enunciado perguntava o que acontece se duas pessoas comprarem o último vaso ao mesmo tempo, e tratei isso: a venda confere e diminui o estoque numa única operação no banco (updateMany com a condição estoque >= quantidade), em vez de ler e gravar em dois passos.

Bônus. Fiz os quatro: busca por nome sem diferenciar maiúsculas (?nome=), ordenação (?ordem=preco ou ?ordem=nome), a rota GET /plantas/esgotadas e um script de seed (yarn seed, arquivo prisma/seed.js) que cadastra cinco plantas de exemplo quando o banco está vazio.

O que ficou de fora. Nenhum item obrigatório nem bônus. As decisões que o enunciado deixou em aberto, ou em que escolhi diferente, estão explicadas na seção "Como foi feito".

2. Como foi feito?

A ordem do trabalho. Segui a ordem do enunciado e fiz um commit ao fim de cada etapa:

1. Ambiente. Conferi que já tinha Git, Node e Docker Desktop, instalei o Yarn e subi o projeto com docker compose up -d --build e yarn migration. No começo trabalhei em

uma branch do repositório original, mas um organizador do processo seletivo recomendou criar o meu repositório a partir do template. Então copiei o que já tinha feito no schema para o repositório novo e refiz o setup, e passei a trabalhar na branch desafio/fcgps.

2. Entender o CRUD de usuário. Antes de escrever código, testei criar, listar, buscar, editar e apagar usuários no Thunder Client e li o UserController e o UserRepository, para entender o caminho de uma requisição: rotas, controller, repository, Prisma e banco.
3. Model e migration. Escrevi o enum Luminosidade e o model Planta no schema.prisma e rodei yarn migration com o nome create_planta_model, o que gerou o SQL da tabela. Li o arquivo gerado, mas não o editei.
4. Repository. Criei o PlantaRepository.ts com uma função por consulta (createPlanta, findAllPlantas, findPlantaById, updatePlanta, deletePlanta), no molde do UserRepository.
5. Controller. Criei o PlantaController.ts com os cinco handlers do CRUD, começando pelo createPlanta, que concentra a maior parte das validações.
6. Rotas. Registrei as rotas no routes.ts e testei cada uma no Thunder Client.
7. Filtros. Acrescentei luminosidade e petFriendly ao GET /plantas.
8. Venda. Criei o POST /plantas/:id/venda.
9. Valor total. Criei o GET /plantas/estoque/valor-total.
10. Concorrência. Reproduzi o problema da venda simultânea com um script e corrigi o repository.
11. Bônus. Fiz GET /plantas/esgotadas, a busca por nome, a ordenação e o seed, um de cada vez.
12. Conferência final. Percorri o checklist do enunciado, inclusive subindo o projeto do zero (docker compose down -v, docker compose up -d --build, yarn migration e yarn seed).

Grupo	Caso	Status
Cadastro	POST /plantas válido, com estoque: 0 e petFriendly: false	201
Cadastro	Sem o campo especie	400
Cadastro	preco: 0	400
Cadastro	estoque: -1	400
Cadastro	preco como texto ("5500")	400
Cadastro	luminosidade: "LUA"	400
Cadastro	petFriendly como texto ("false")	400
Cadastro	regaACadaDias: 0	400
Buscar	GET /plantas/1 (existe)	200
Buscar	GET /plantas/9999 (não existe)	404

Buscar	GET /plantas/abc	400
Editar	PATCH só com o preço (o resto não muda)	200
Editar	PATCH com estoque: -5	400
Editar	PATCH em id inexistente	404
Filtros	?luminosidade=SOMBRA	200
Filtros	?petFriendly=true	200
Filtros	?luminosidade=MEIA_SOMBRA&petFriendly=true	200
Filtros	Filtros válidos sem nenhuma planta que combine	200, []
Filtros	?luminosidade=LUA	400
Filtros	?petFriendly=abc	400
Venda	quantidade: 2 (estoque 5 passa a 3)	200
Venda	quantidade: 10, maior que o estoque (o estoque continua 3)	400
Venda	quantidade: 0	400
Venda	quantidade como texto ("2")	400
Venda	Corpo {} (sem quantidade)	400
Venda	Id inexistente	404
Valor total	Com as cinco plantas do seed (valorTotalEmCentavos: 165380, totalDeVasos: 35, conferido na mão)	200
Valor total	Loja vazia (valores zerados)	200
Deletar	DELETE /plantas/5	204
Deletar	GET /plantas/5 depois de deletar	404
Deletar	DELETE /plantas/5 de novo	404
Bônus	GET /plantas/esgotadas	200
Bônus	?nome=COSTELA (ignora maiúsculas)	200
Bônus	?nome= vazio	400
Bônus	?ordem=preco	200
Bônus	?ordem=abc	400

Concorrência	Cinco vendas simultâneas de uma planta com estoque 1: um 200 e quatro 400, estoque final 0	200 e 400

Decisões de validação, e o porquê.

- === undefined em vez de !campo. Para saber se um campo foi enviado, comparo com undefined. Com !campo, um estoque 0 e um petFriendly: false seriam recusados, e os dois são valores válidos.
- Number.isInteger para números. Ele recusa texto como "8990", decimais como 89.9 e NaN, que o enunciado manda tratar como 400. O mesmo vale para o id da URL (que chega como texto) e para a quantidade da venda.
- Texto, booleano e enum. Uso typeof com trim() para textos não vazios, typeof petFriendly === "boolean" e Object.values(Luminosidade).includes(...) para o enum.
- Buscar antes de editar, apagar ou vender. O update e o delete do Prisma lançam erro quando o id não existe, e esse erro cairia no catch como 500. Buscando a planta antes, respondo 404, como o enunciado pede.
- PATCH só valida o que foi enviado. Cada campo passa pelas mesmas regras do cadastro, mas só se vier no corpo (campo !== undefined && ...). Os campos undefined são ignorados pelo Prisma, então editar só o preço não mexe no resto.
- Nunca passo o req.body direto ao banco, envio só os campos da planta.
- Filtros. Os valores de req.query chegam como texto, então valido o luminosidade e aceito só "true" ou "false" no petFriendly, e respondo 400 para o resto. Converto o petFriendly para booleano só quando o filtro foi enviado, para um filtro ausente não virar false e listar só as plantas sem pet-friendly. Uma única consulta com where resolve todas as combinações, porque o Prisma ignora condições undefined. Filtro válido sem resultado responde 200 com [], e não 404.

Decisões de organização.

- Camadas. Mantive a mesma divisão do CRUD de usuário: rotas ligam a URL à função, o controller lê, valida e escolhe o status, e só o repository fala com o Prisma.
- Venda como POST, não PATCH. Uma venda não é uma edição de campos da planta, é uma operação com regras próprias (conferir o estoque, recusar quando não há). Com o PATCH, o cliente teria que calcular o novo estoque e enviá-lo, o que piora o problema das compras simultâneas.
- Venda sem estoque negativo. O handler valida o id e a quantidade, busca a planta (404), confere o estoque (400, sem alterar nada), diminui e responde 200 com a planta atualizada. No banco, a diminuição é um único updateMany com a condição estoque >= quantidade e decrement, e o count do resultado diz se a venda aconteceu.
- Valor total. Como o aggregate do Prisma não soma preco × estoque, busco só esses dois campos com select e faço a conta com um for...of. Com a loja vazia, o laço não roda e a resposta sai zerada.
- Ordem das rotas. As rotas fixas (/plantas/esgotadas, /plantas/estoque/valor-total) ficam antes das que têm :id, porque o Express usa a primeira rota que combina.
- Commits. Um por etapa, com mensagens que dizem o que mudou.

Onde o enunciado deixou em aberto.

Busca por nome. Uso contains com mode: "insensitive". Um ?nome= vazio responde 400, para não listar tudo sem aviso. A busca ignora maiúsculas.

Ordenação. O enunciado não diz a direção, então ordeno de forma crescente. Um valor diferente de preco ou nome responde 400.

Seed. Escrevi em JavaScript (prisma/seed.js, rodado por yarn seed) para não adicionar uma dependência só para esse bônus. Ele só cadastra as cinco plantas se o banco estiver vazio, para não duplicar nada ao rodar duas vezes.

JSON inválido. Um corpo com JSON malformado é recusado pelo Express antes de chegar ao meu código, com um 400 em HTML. O enunciado não pede para tratar isso e eu não mexi.

3. As principais dificuldades

Começar do zero. Eu não tinha experiência com Docker, CRUD, Git nem TypeScript, e a maior dificuldade inicial foi entender o caminho de uma requisição (rotas, controller, repository, Prisma e banco) e o que o yarn migration faz. Para destravar, testei o CRUD de usuário no Thunder Client e li o código dele antes de escrever o meu, e pedi explicações quando um conceito não ficava claro.

Repositório e pasta. Comecei trabalhando numa branch do repositório original e, quando um organizador recomendou o template, precisei refazer o setup num repositório novo. O git clone falhou com destination path ... already exists porque a pasta antiga tinha o mesmo nome, e o Remove-Item foi bloqueado pelo Windows porque a pasta estava em uso. Depois de rodar docker compose down e fechar o que estava aberto nela, consegui apagar e clonar de novo. Antes de apagar, copiei o model da Planta, para não perder o que já tinha feito.

Erro no schema. Escrevi Bool em vez de Boolean no campo petFriendly. Esse tipo não existe no Prisma, e a migration só funcionou depois de corrigir.

API fora do ar nos testes. O Thunder Client mostrou Connection was refused by the server. A causa era o Docker Desktop fechado, que derruba os containers. Abri o Docker Desktop e rodei docker compose up -d.

Tipos do TypeScript. O req.query tem um tipo largo (texto, lista ou objeto), e o editor recusou passar a luminosidade do filtro ao includes do enum. Resolvi com um “as Luminosidade”, que é seguro porque a validação já recusa valores fora do enum.

Erros de lógica que os testes e a revisão pegaram.

- No valor total, usei for (planta in plantas) e somei preco + estoque. O for...in percorre os índices, e não os itens, e a conta certa é preco * estoque. Corrigi para for...of e *, e a conta feita na mão confirmou o resultado.
- Na venda, validei a quantidade só com quantidade <= 0, o que deixava passar campo ausente, texto e decimais. Troquei por Number.isInteger(quantidade) && quantidade > 0.
- Depois de trocar a venda por updateMany, chamei a função de venda duas vezes no controller (a segunda deveria ser uma busca), o que diminuiria o estoque em dobro. Corrigi para buscar a planta atualizada.

Ordem das rotas. Depois de criar GET /plantas/esgotadas, a chamada respondia 400 com a mensagem do id, em vez de uma lista. Entendi que o Express usa a primeira rota que combina, e que esgotadas estava caindo em GET /plantas/:id, onde Number("esgotadas") dá NaN. Movi as rotas fixas (/plantas/esgotadas e /plantas/estoque/valor-total) para antes das rotas com :id, e passou a funcionar.

Duas compras ao mesmo tempo. Era a pergunta do enunciado sobre o último vaso. Para ver o problema acontecer, criei uma planta com estoque 1 e escrevi um script

(teste-concorrencia.js, que eu deletei antes de dar commit) que dispara cinco vendas simultâneas. O resultado foi cinco respostas 200 e o estoque em -4, ou seja, todas passaram pela conferência antes de qualquer uma diminuir o estoque. A causa é que conferir e diminuir eram dois passos separados. Corrigi trocando a venda por um updateMany que só diminui se estoque >= quantidade, tudo numa operação no banco, e passei a usar o count do resultado para saber se a venda aconteceu.

4. O uso da IA

Usei o Claude ao longo de todo o desafio.
Para que usei.

- Entender conceitos: o que são API, CRUD, rotas e status HTTP; a diferença entre Docker, Docker Compose e Docker Desktop; o que cada comando do enunciado faz (yarn, docker compose up, yarn migration); e o papel de rotas, controller e repository.
- Planejar: a ordem das etapas, o roteiro de alguns handlers e as ferramentas de validação.
- Revisar o meu código: para ver se eu deixei passar algum erro.

O que escrevi eu mesmo, usando o UserController e o UserRepository como modelo.

- O PlantaRepository com o CRUD básico.
- As validações do createPlanta e do updatePlanta, o readPlantaById, o readAllPlantas sem filtros, o deletePlanta e as rotas do CRUD.
- O model e o enum no schema.prisma.

O que escrevi com ajuda (a IA deu o roteiro, e eu preenchi).

- O handler da venda, com a ordem dos passos dada pela IA e as condições escritas por mim.
- O valor total, a partir de um esqueleto com a parte do laço em branco.
- As mensagens de erro das validações, a partir de exemplos.

O que veio da IA e eu adaptei.

- O código dos filtros no readAllPlantas, da busca por nome e da ordenação.
- A troca da venda por updateMany para tratar a concorrência.
- O script de seed em JavaScript.

Erros que a revisão da IA apontou no meu código: Bool no lugar de Boolean, o for...in com + no valor total, a validação incompleta da quantidade na venda, a venda chamada duas vezes.
Como conferi o que a IA respondeu.

- Rodei yarn typecheck depois de cada etapa.
- Testei cada rota no Thunder Client, inclusive os casos de erro, e conferi no Prisma Studio o que ficou gravado no banco.
- Comparei o que ela sugeria com o UserController e com as dicas do próprio enunciado.
- Conferi a conta do valor total na mão.
- Reproduzi o problema da concorrência antes de corrigi-lo, em vez de aceitar a explicação, e refiz o teste depois.

O que aprendi. Como uma requisição percorre rotas, controller, repository e banco; por que validar com === undefined e não com !campo; a diferença entre 400, 404 e 500; por que a ordem das rotas importa no Express; o que é uma migration e por que não se edita uma já aplicada; e o que é uma condição de corrida e como resolvê-la fazendo a conferência e a escrita numa operação só no banco.